CS636 Formal Methods

Topics Covered

Introduction to Formal Methods

Software Development Life Cycle (SDLC)

Software Development Life Cycle (SDLC) is a systematic approach to developing software applications.

- **Software increasing in size and complexity**
- ***A Systematic approach is required called SDLC***

Question

- ***Requirement Specification***
- ***Design***
- ***Coding***
- ***Deployment***

Software Errors

- **Increased size and complexity leads to software errors**
  - ***Error: software does not provide the intended behavior***
- ***Errors lead to failure***
  - ***Cost***
- ***High relieability software cannot tolerate errors***
  - ***Incorrect specifications OR Specification were not followed (misinterpreted)***
  - ***Specifications are important for error free software***

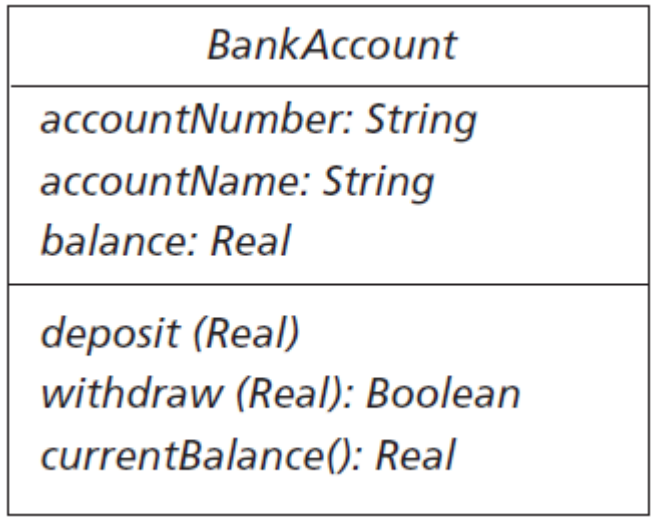
Software Specifications

- **Correctness of entire software depends on specification**
  - ***Ripple affect***
- ***Currently written in natural language and diagrammatic notations (UML for example)***
  - ***Ambiguous***
  - ***Incomplete***
  - ***Inconsistent***

Software Specifications

Does withdraw mean

A cash withdrawal refers to taking money out of a bank account



A typical UML diagram for the *BankAccount* class *

Withdraw: R*eceives a requested amount to withdraw from the bank account and, if there are sufficient funds in the account, meets the request. Returns a boolean value indicating success or failure of the attempt to withdraw money from the account*.

- Is it that the bank account must contain *at least* the amount of money that Is requested for withdrawal?
- Or is there a minimum balance that must be maintained?
- Or is there an agreed overdraft limit?

* Formal Software Development, from VDB to Java by Quentin Charatan and Aaron Kans

Software Specifications

- **Specification of “Withdraw” method is:**
  - **Ambiguous (different interpretations)**
  - **Incomplete (What should happen when there is insufficient amount)**
  - ***Inconsistent (Overdraft facility may be defined somewhere else)***

Software Testing

Drawback of testing

- **Testing aims to locate the errors**
  - ***Testing cannot take place until some implementation is available***
  - ***Correcting errors identified by testing may introduce new errors***
  - ***Testing can uncover errors, does not guarantee absence of them***
  - ***Testing is based on specifications, ambiguous specifications may make testing ineffective***

Formal Notations

Formal notations are symbols, numbers, and letters used in a structured way to express mathematical or logical ideas clearly.
They follow strict rules to avoid confusion and make it easier to solve problems, prove theories, or represent information

- **It is desirable to use a specification notation with a fixed, unambiguous semantics.**
- **Notations that have a fixed semantics are known as formal notations, or formal languages.**
- **A fixed semantics is achieved by defining a language in a completely unambiguous way using a mathematical framework.**

Formal Methods

Formal methods are structured, mathematical approaches used to design and verify systems, like software or hardware, to ensure they work correctly and without errors

- **Formal methods are methods that use formulas. A formula is a text or diagram constructed from predefined symbols combined according to explicit rules.**

- **A good working definition of formula is anything whose appearance or syntax can be checked by a computer.**
- ***Z language***

Formal Methods

- **Formal methods** combined **constitute a branch of software engineering that incorporates the use of mathematics for software development.**
- **Formal methods use formula. A good working definition of formula is anything whose appearance or syntax can be checked by a computer.**

Advantages of formal methods

- Formal specifications can help considerably in generating suitable test cases.
- Formal specifications allow for feedback on system specifications at early development stages
- Important properties (such as internal consistency) of the initial specification can be checked mathematically
- A proof of program correctness can be constructed
- Proofs can help uncover design errors as soon as they are made

Thank You

CS636 Formal Methods

Topics Covered

Introduction to Formal Methods

When are the formal methods useful?

- **Novel Projects: Novel Projects are about building something new, similar of which does not exist before.**
  - **Compare alternative ides**
  - **Select the best one (through in-depth analysis)**
  - **Because we cannot afford building the alternative versions**
- **Difficult Project: Projects are difficult when they address something profound or deep**
  - **May not need to be long**
  - **A simple page of code can be too difficult**

When are the formal methods useful?

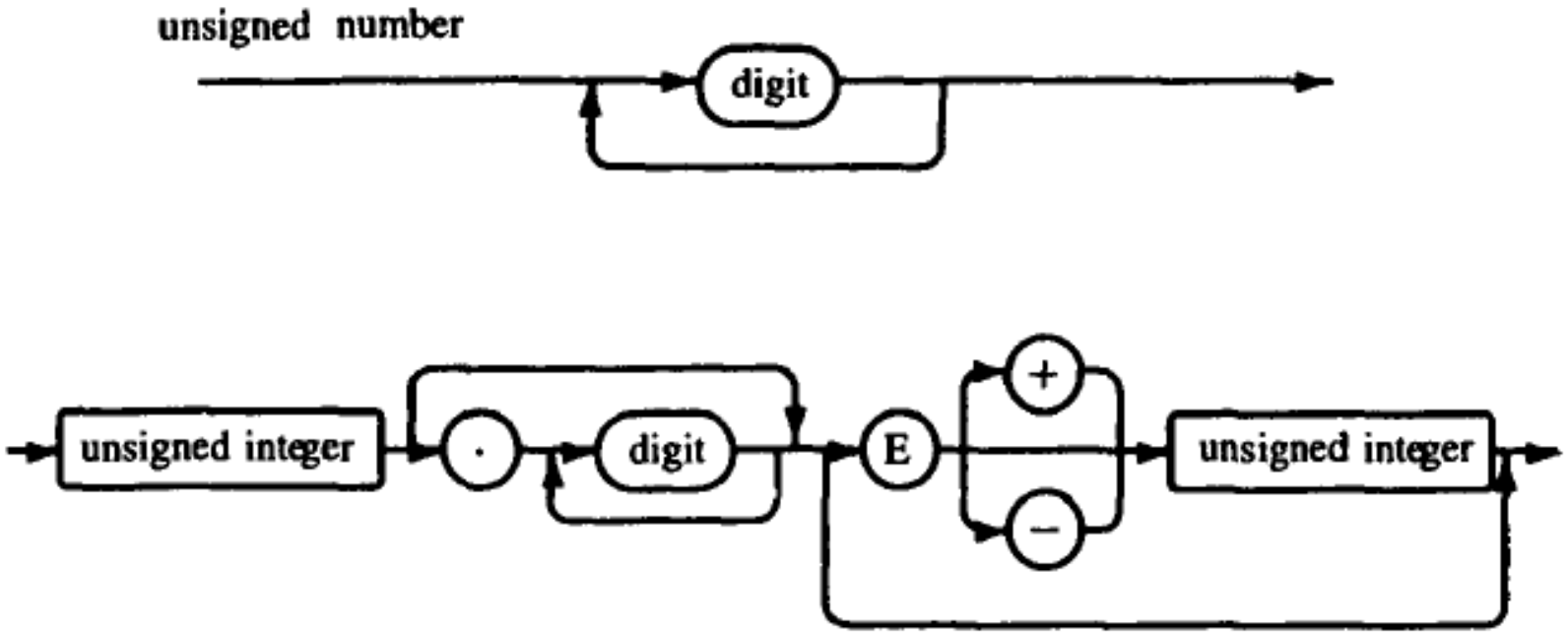
- **Critical Projects: So much is at stake**
  - **Money**
  - **Security**
  - **Safety**
- **You may not need to use formal methods for simple and routine jobs which are obvious and too easy.**

How can we use formal Methods?

- **Modeling**
  - **Models describe the actual product behavior (before implementing it) through some symbols/notations etc.**
  - **A model is a simplified representation. It may skip the irrelevant properties**
  - **A mathematical model that represents the intended behavior of a program can be used as a formal specification**

A formal model of how to form a number

- Specification permits 1, 0.5, 1E5, and 1.5E-5
- prohibits 1., .5, E5, 1. 5E-5.0



* Jensen, Kathleen and Wirth, Nicklaus. *PASCAL User Manual and Report.* Springer-Verlag, Berlin, 2nd ed., 1974.

How can we use formal Methods?

- **Design**
  - **Top Down**
  - **Bottom Up**
  - **Z Language**

How can we use formal Methods?

- **Verification**
  - **The code does what it is intended for**
  - **A convincing product of verification is a proof**
  - **Proof provides greater confidence than testing**

Thank You

CS636 Formal Methods

Topics Covered

Mathematical Notations

Notations and descriptions

$\mathbb{N}$	*Natural Numbers*	1,2,3,4,5....n
$\mathbb{Z}$	*Integers*	+- 0,1,2,3,4,5.....n
$\forall$	*For all*	$\forall$ x, x+1>x = 1+1>2
$\exists$	*There exists*	$\exists$ x : x=x2 1=1
$\nexists$	*There does not exist*	x , x= x+1 0=0+1
$\wedge$	*Logical AND*	Both True
$\vee$	*Logical OR*	One true & one false

CS636 Formal Methods

Topics Covered

Mathematical Models

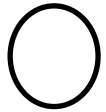
Petri nets

- Introduced by Carl Adam Petri in 1962
- Diagrammatic tool to model concurrency and synchronization
- Provides mathematical modeling as well

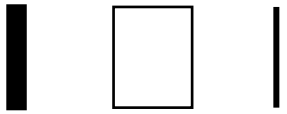
Petri nets

QIUZ

- Tow main components are Places and transitions.
- Place (Represents condition or State) denoted by Circle



- Transition (Represents Events) denoted by Rectangle, Vertical bar or filled vertical rectangle



- Arc (connect Transitions and Places) denoted by directed arrow

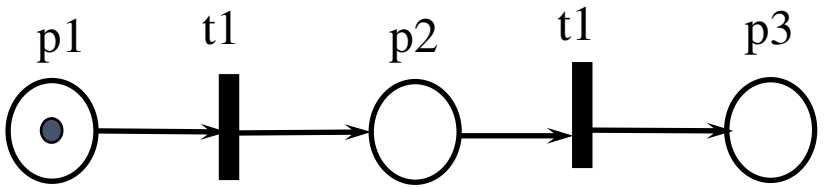


- Tokens (represents state at time t) denoted by a filled circle

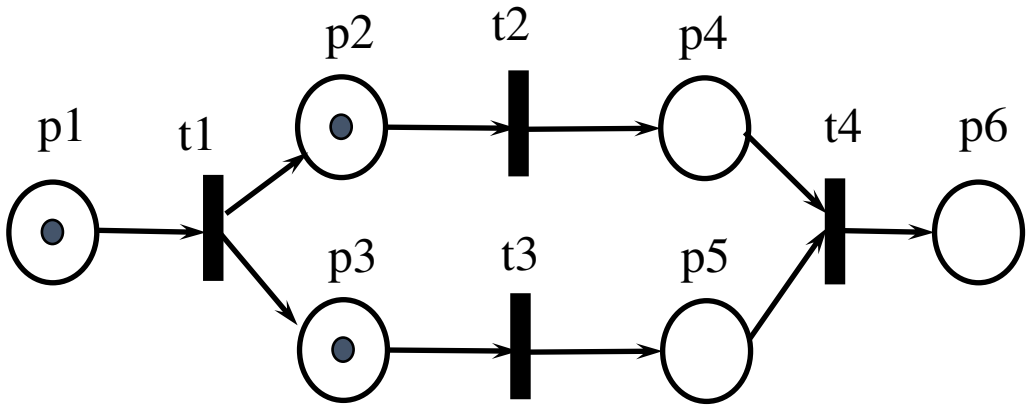


- Each transition has an input place and an output place. Input place is also called pre-condition and output place is also called post condition
- For a transition to fire, there should be at least one token at each input place
- Once a transition is fired, one token is removed from all of its input places and one token is pasted in all of its output places

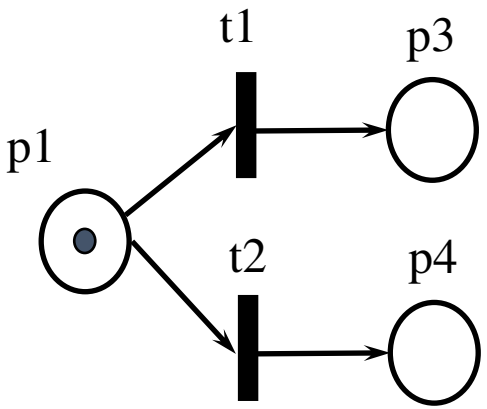
- Sequence



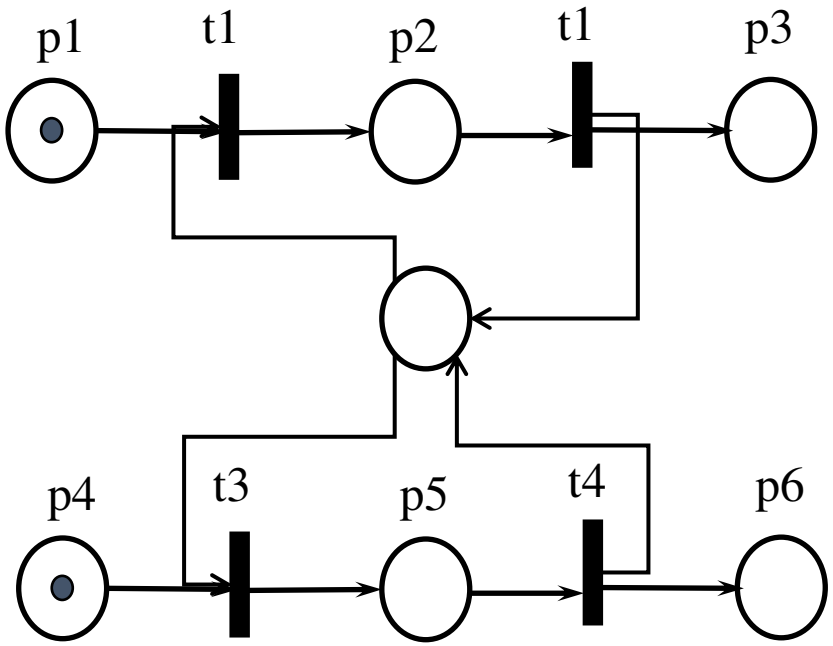
- Concurrency



- Conflict



Resource Sharing

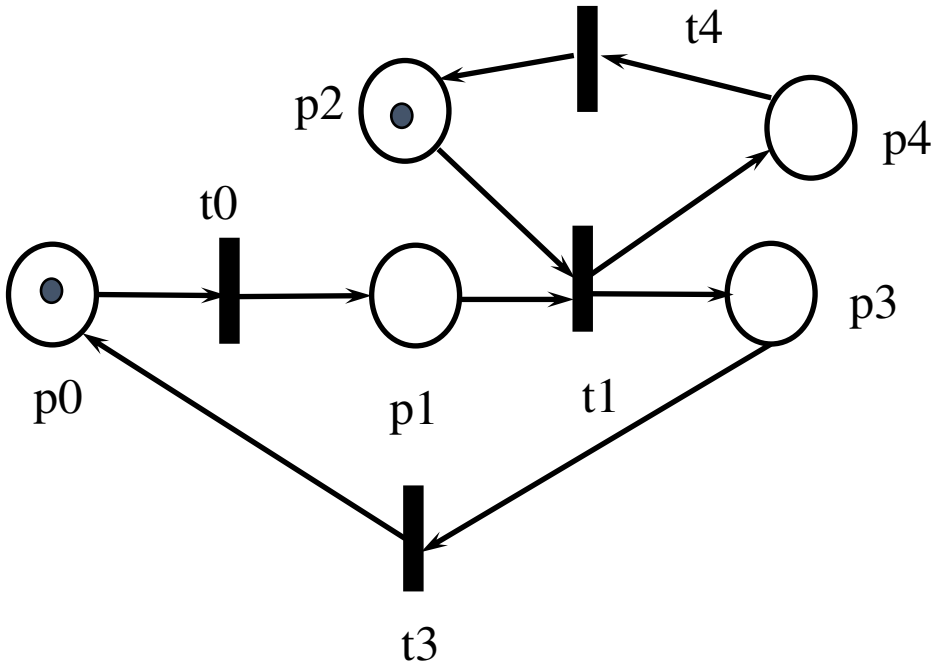


CS636 Formal Methods

Topics Covered

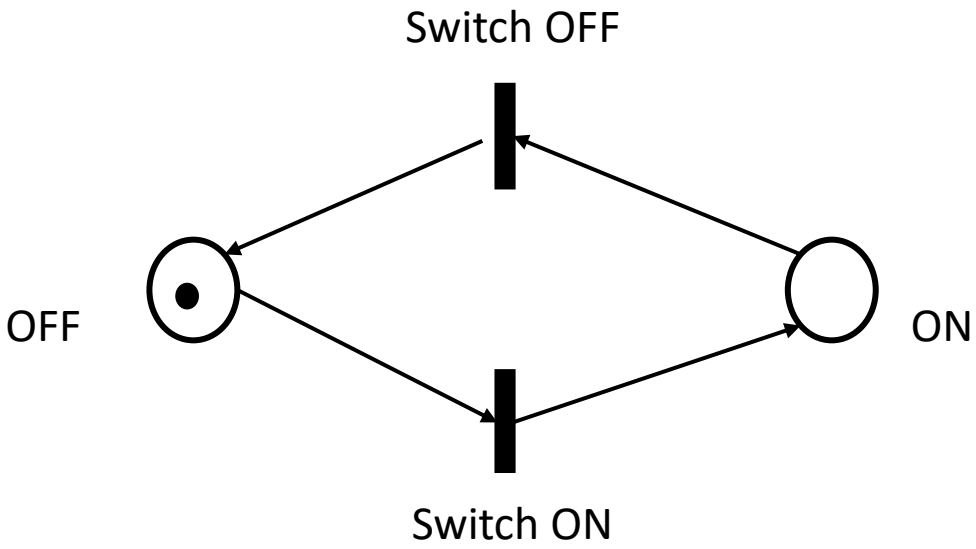
Formal Specification Techniques

Petrinets

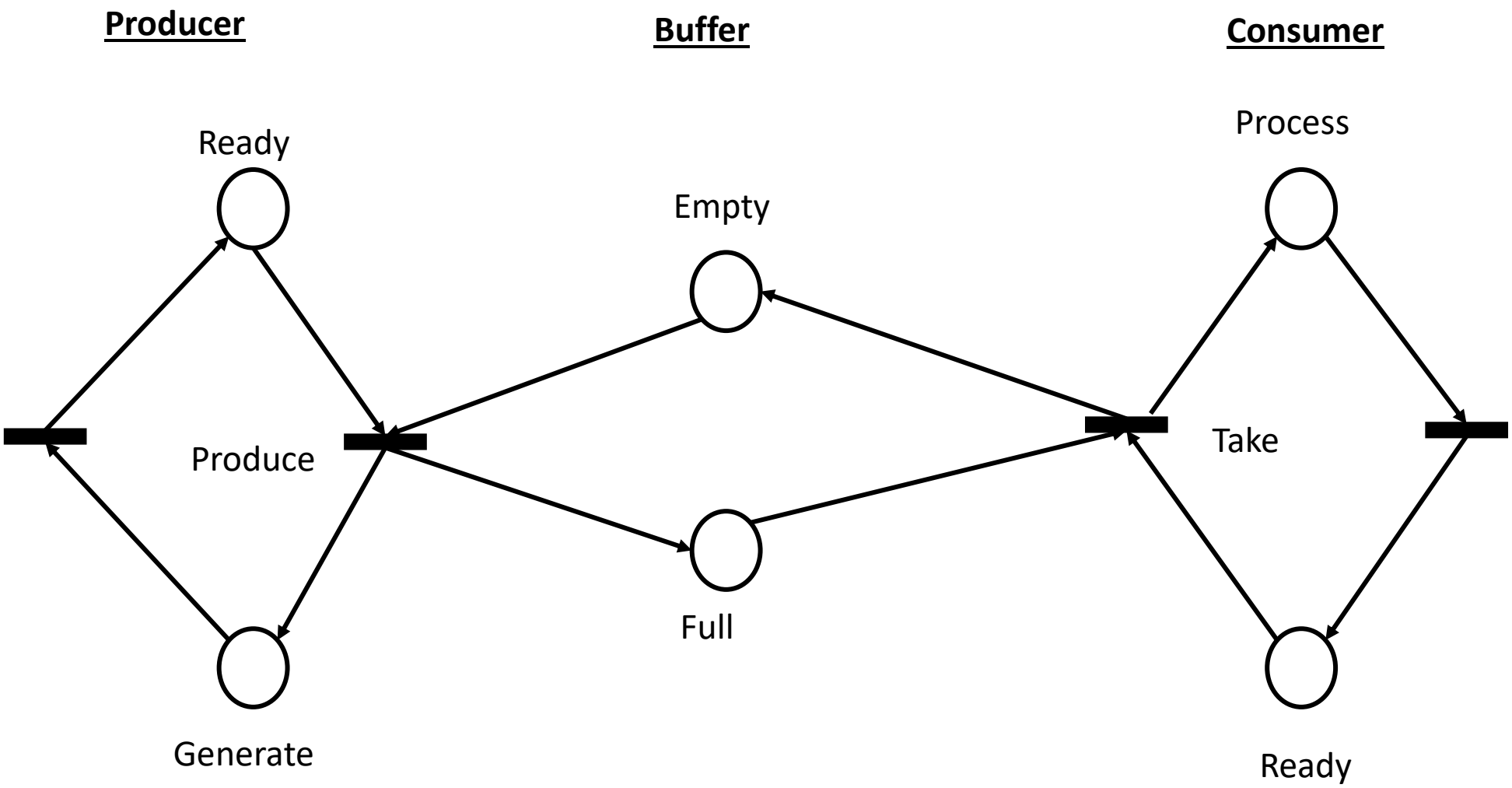


- A transition can be fired when there is at least one token on each of the transition's input places
- When a transition is fired it one token is removed from each of its input places and produced a single token on each of its output places.
- A transition that is able to fire is said to be enabled and otherwise disabled.
- If there is more than one enabled transition any one of enabled transitions may be the next one to fire.

Petri net: A simple example



Producer Consumer Problem



CS636 Formal Methods

Topics Covered

Mathematical Models

Formal Definition

- Petri net PN = (P,T,I,O)

- P = Set of Places
- T = Set of Transitions
- I = Inputs
- O = Outputs

Marking

- Marking *M* represents the state of a petri net

marking M show the current state of a petri net

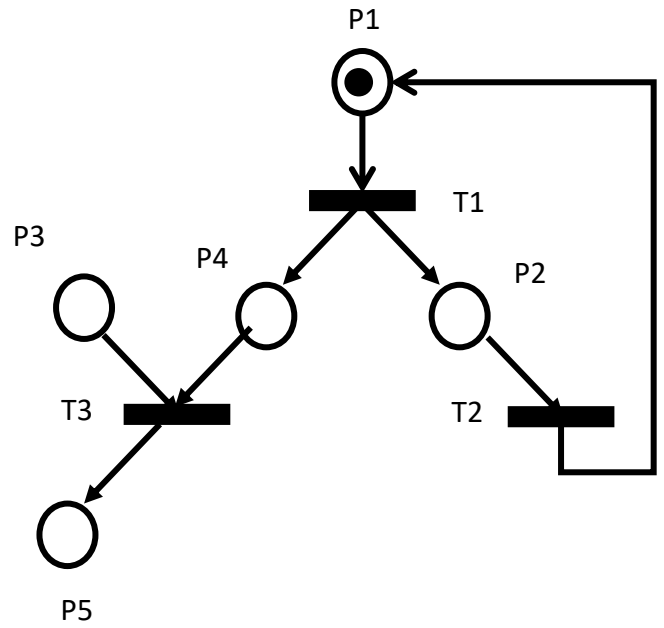
Reachability

- A Marking M_i is reachable from M_o if there exists a firing sequence that transforms M_o to M_i

Boundedness

Number of tokens infinite ni hony chaiye

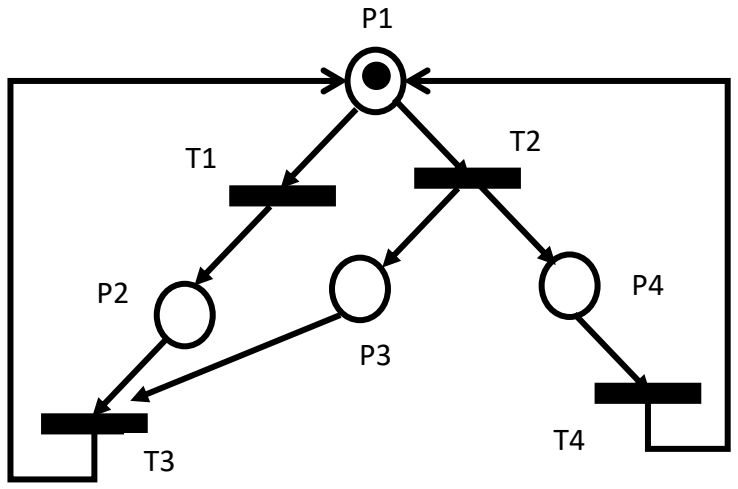
- A Petri net is K bounded if the number of tokens in any place does not exceed K for any marking reachable from M_o



Liveness

Asi state ni ani chaiye jis mn ap mn ap koi b transition fire na krwaa saky

- A Petri net with initial marking M_0 is live if for any Marking M_i (after M_0) we can still fire a transition.
- If a petri net is live, it is deadlock safe



CS636 Formal Methods

Topics Covered

Automata

Automata

state user ki inputs k sth change hoti rehti

- Automata are abstract mathematical models of machines that perform computations on an input by moving through a series of states or configurations.

Example: Like elivator in shopping mall

Language

- Language describes the conditions imposed on its words
- The language L of strings of odd length, defined over Σ={a}, can be written as L={a, aaa, aaaaa,.....}

Kleene Star Closure

- Given Σ, then the Kleene Star Closure of the alphabet Σ, denoted by Σ* , is the collection of all strings defined over Σ, including Λ.
- Examples:
- If Σ = {x} Then
- Σ* = {Λ, x, xx, xxx, xxxx, ….}
- If Σ = {0,1} Then Σ* = {Λ, 0, 1, 00, 01, 10, 11, ….}
- If Σ = {aaB, c} Then Σ* = {Λ, aaB, c, aaBaaB, aaBc, caaB, cc, ….}

PLUS operation

- Plus Operation is same as Kleene Star Closure except that it does not generate Λ (null string), automatically.
- Example
- If Σ = {0,1} Then Σ^+ = {0, 1, 00, 01, 10, 11, ….}
- If Σ = {aab, c} Then Σ^+ = {aab, c, aabaab, aabc, caab, cc, ….}

Regular Expression

- As discussed earlier that a* generates Λ, a, aa, aaa, … and
- a+ generates a, aa, aaa, aaaa, …,
- so the language
- L1 = {Λ, a, aa, aaa, …} and L2 = {a, aa, aaa, aaaa, …} can simply be expressed by a * and a+ , respectively.
- a * and a+ are called the regular expressions (RE) for L1 and L2 respectively

Regular Expression

- Every letter of Σ including Λ is a regular expression.
- If r1 and r2 are regular expressions then
- (r1)
- r1 r2
- r1 + r2 and
- r1* are also regular expressions.

CS636 Formal Methods

Topics Covered

Automata

Regular expressions

- a(a+b)*a
- (aa+bb)(ab)
- b(aa+bb)$^{+}$

Finite Automata

- Finite automata is a state machine that takes a string of symbols as input and changes its state accordingly.
- A string is accepted if it reaches final state
- We will discuss two automat
  - Deterministic Finite Automata
  - Non Deterministic Finite Automata

Deterministic Finite Automata

- **A DFA is a 5-tuple, (S, ∑, T, s, A)**
  - **S represents finite set of states**
  - **∑ represents set of input symbols**
  - **T set of transitions**
  - **s ∈ S represents initial state**
  - **A ∈ S repreents final state**

Deterministic Finite Automata

- S = {a, b, c},
- ∑ = {0, 1},
- s = {a},
- A = {c}

Current State	Next State for input 0	Next state for input 1
a	b	a
b	a	c
c	b	c

Non Deterministic Finite Automata

- **A DFA is a 5-tuple, (S, ∑, T, s, A)**
  - **S represents finite set of states**
  - **∑ represents set of input symbols**
  - **T set of transitions**
  - **s ∈ S represents initial state**
  - **A ∈ S repreents final state**

Non Deterministic Finite Automata

- S = {a, b, c},
- ∑ = {0, 1},
- s = {a},
- A = {c}

Current State	Next State for input 0	Next state for input 1
a	a,b	b
b	a	a, c
c	b,c	c

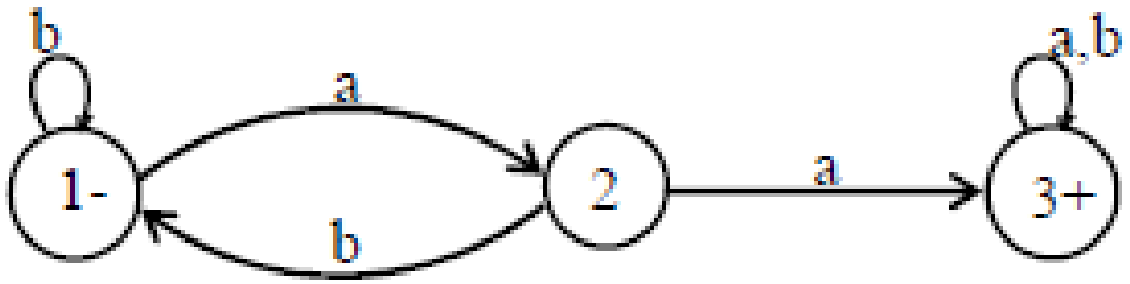
CS636 Formal Methods

Topics Covered

Automata

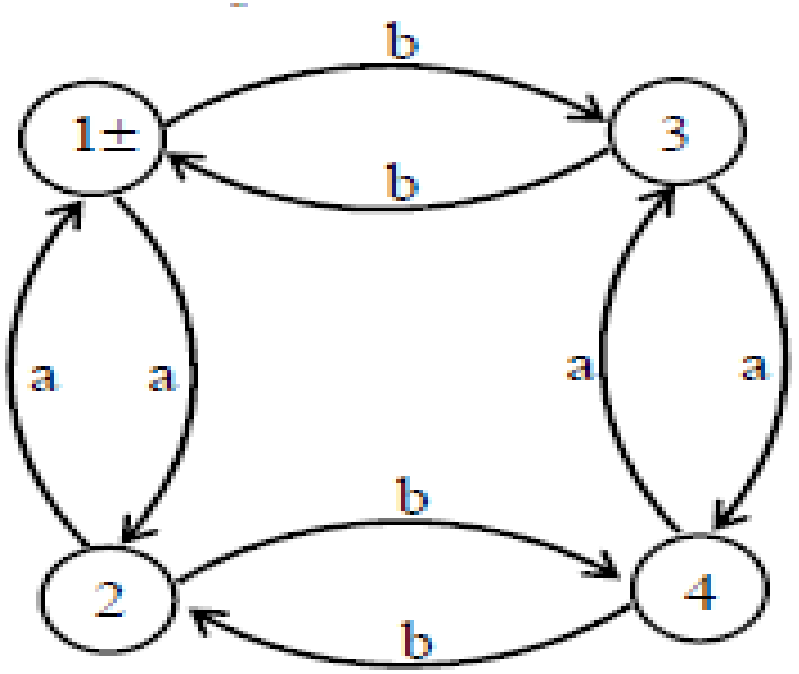
Deterministic Finite Automata: Example

- Consider the Language L of strings , defined over Σ = {a, b}, containing double a.
- RE: (a+b)*aa(a+b)*



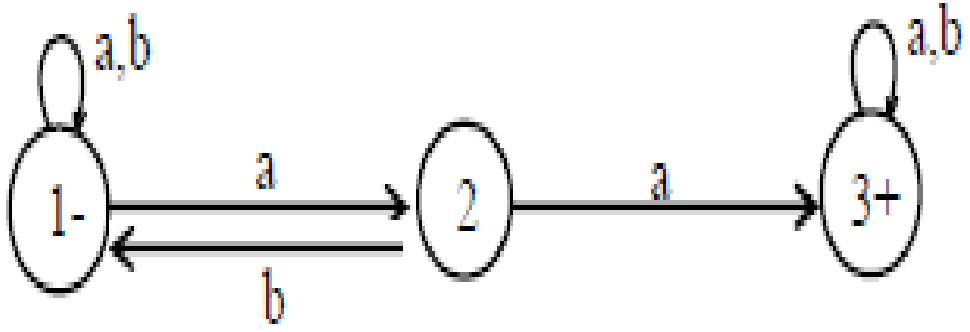
Deterministic Finite Automata: Example

- Consider the EVEN-EVEN language, defined over Σ = {a, b}.
- (aa+bb+(ab+ba)(aa+bb)*(ab+ba))*



Non Deterministic Finite Automata: Example

- Consider the Language L of strings , defined over Σ = {a, b}, containing double a.
- RE: (a+b)*aa(a+b)*



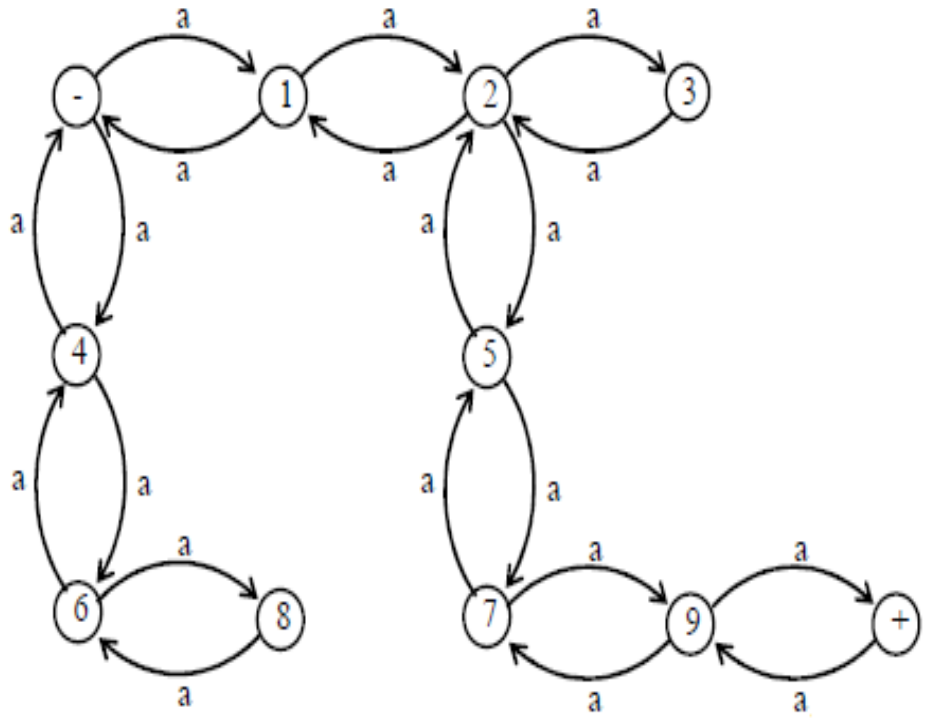
Non Deterministic Finite Automata: Example

- Maze Game
- - and + indicate the initial and final states respectively. One can move only from a box labeled by other then L, M, N, O, P to such another box.

-	1	2	3
4	L	5	O
6	M	7	P
8	N	9	+

Non Deterministic Finite Automata: Example

-	1	2	3
4	L	5	O
6	M	7	P
8	N	9	+



Deterministic Finite Automata: Example

- A Vending Machine
- Accepts $1 and $0.25 per soda
- Once machine receives $1.25, it allows user to select soda
- Self-loop represents ignored input
- Soda will not be dispensed until $1.25 is deposited
- Machine will not accept more money, once $1.25 is deposited

S = {$0.00; $0.25; $0.50; $0.75; $1.00; $1.25; $1.50; $1.75; $2.00}

$\sum$ = {$0.25; $1.00; select}

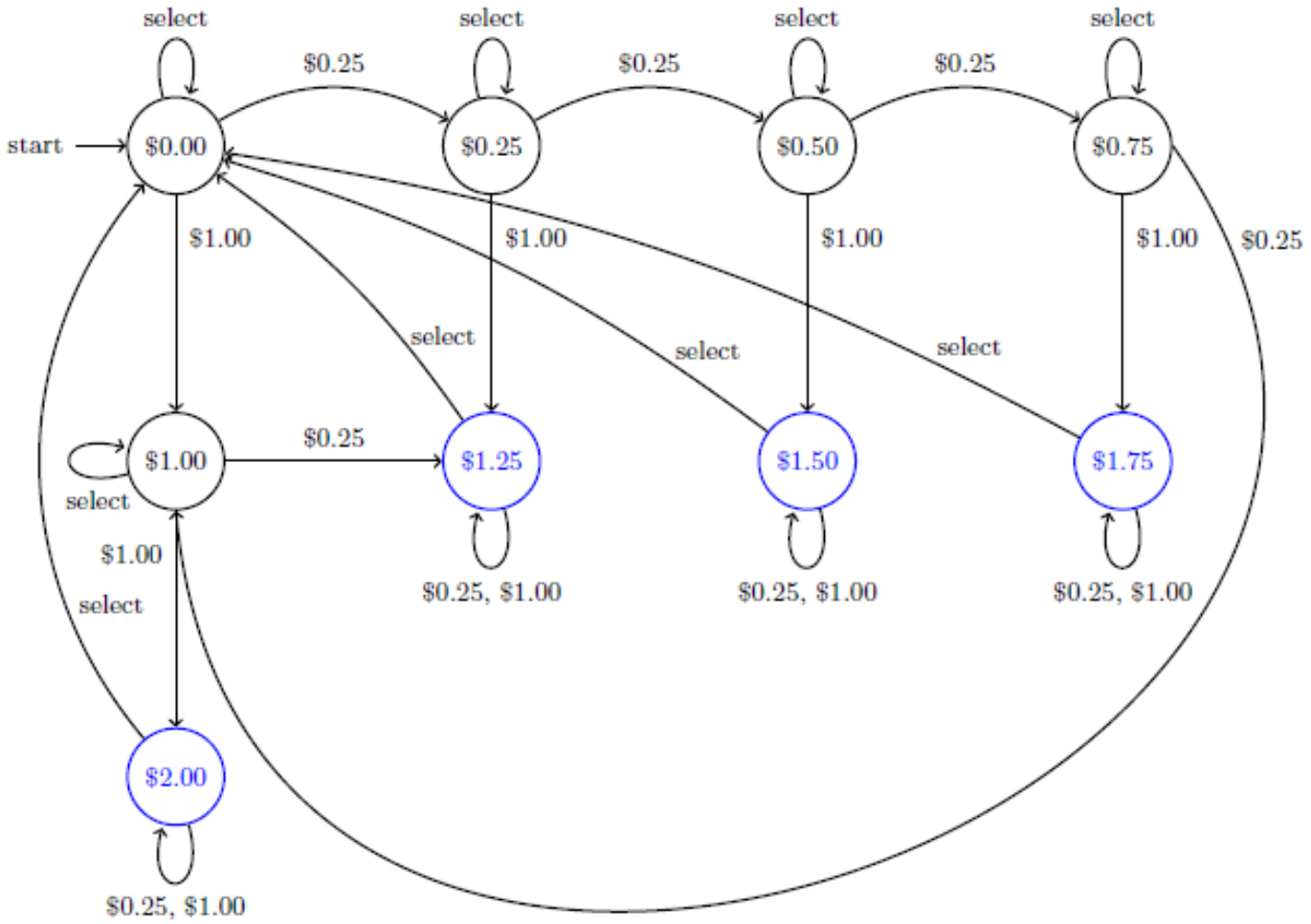
The transition function, is described by the state diagram.

s= $0.00 is the start state

A $\in S$ is the set of accept states

Source: Gribkoff, E. (2013). Applications of deterministic finite automata. *UC Davis*, 1-9.

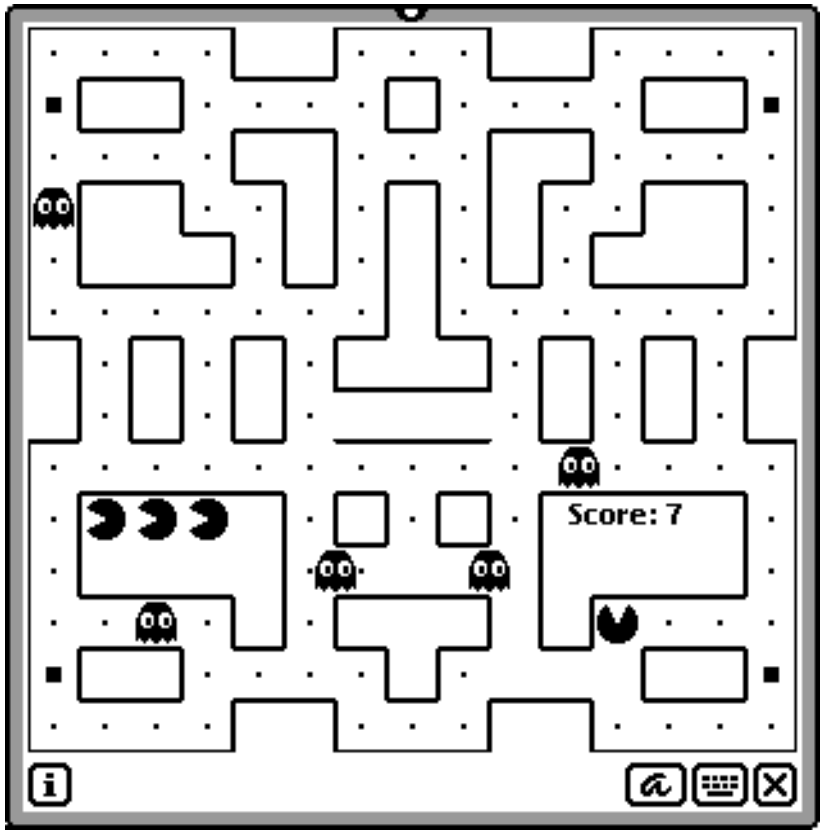
Deterministic Finite Automata: Example



Source: Gribkoff, E. (2013). Applications of deterministic finite automata. *UC Davis*, 1-9.

Deterministic Finite Automata: Example

- Pac-Man's Ghosts
- Player Navigates through the maze, eating pellets and avoiding ghost.
- If Pac-Man eats power pellet, it can eat ghosts.
- Once a Pac-Man eats power pallet, the ghost try to avoid Pac-Man

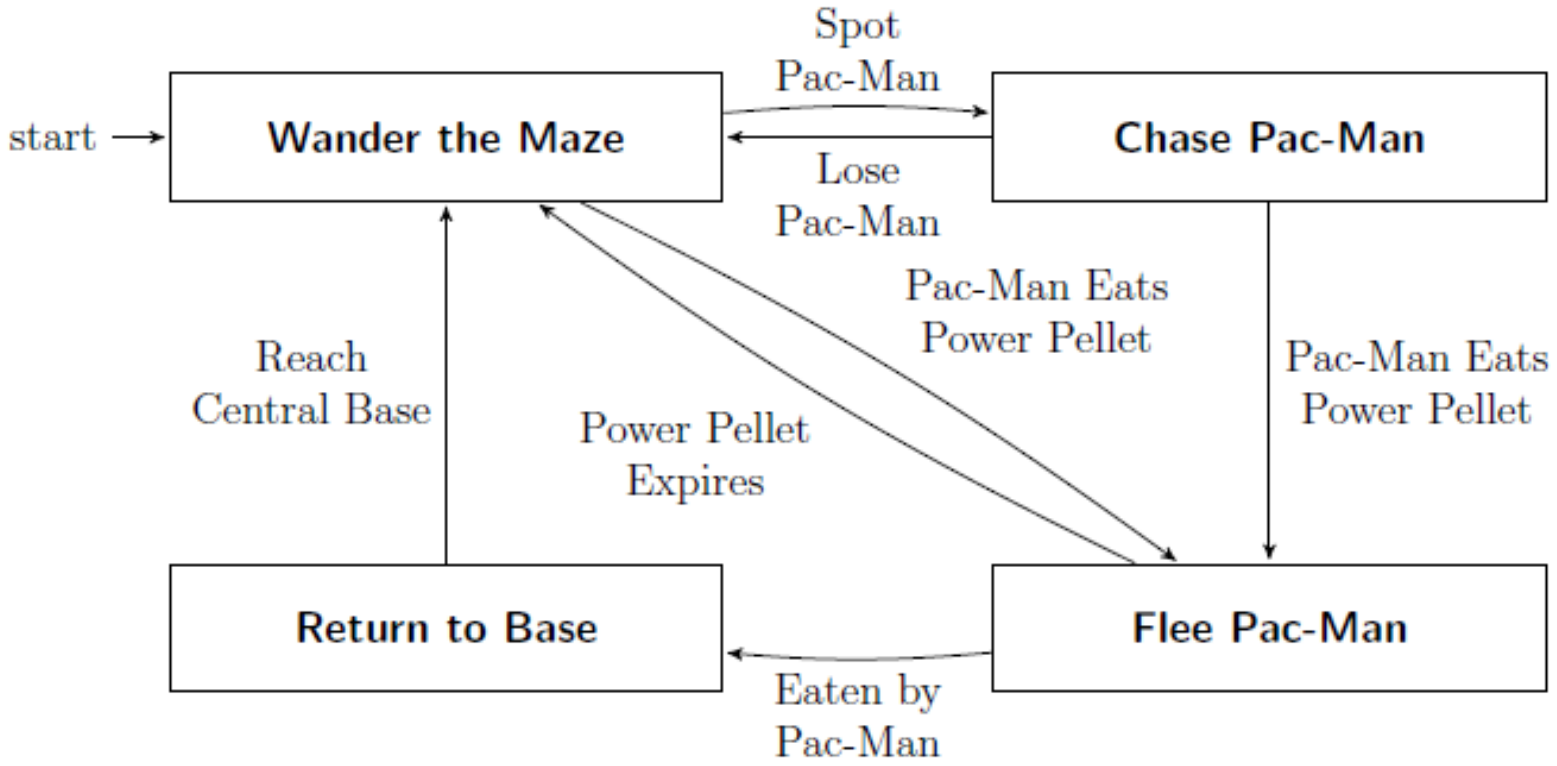


Source: Gribkoff, E. (2013). Applications of deterministic finite automata. *UC Davis*, 1-9.

Deterministic Finite Automata: Example

- **Behavior of Ghosts:**
- 1. Randomly wander the maze
  line of sight
- 3. Flee Pac-Man, after P
  regenerate

2. Chase Pac-Man, when he is within



Source: Gribkoff, E. (2013). Applications of deterministic finite automata. *UC Davis*, 1-9.

CS636 Formal Methods

Topics Covered

Formal Verification

References:

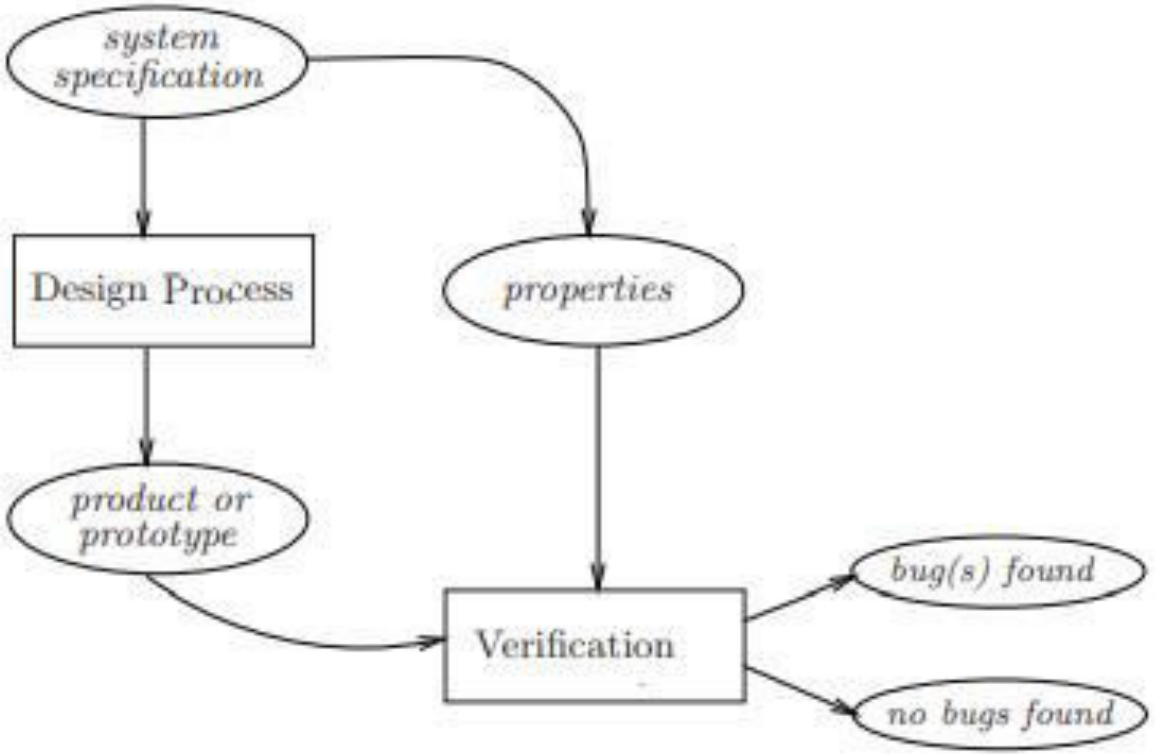
All the contents in these slides are taken from:

- Baier, Christel, and Joost-Pieter Katoen. *Principles of model checking*. MIT press, 2008.

System Verification

- **The reliability of ICT systems is a key issue in the system design process**
- **Reliability verification:**
  - ***Software System***
  - ***Hardware System***
  - ***Concurrent System***

System Verification



Verification of the Models

- **Model-based verification techniques are based on models describing the possible system behavior in a mathematically precise and unambiguous manner.**
- ***The accurate modeling of systems often leads to the discovery of incompleteness, ambiguities, and inconsistencies in informal system specifications***

Verification of the Models

remain with algorithem

- **The system models are accompanied by algorithms that systematically explore all states of the system model.**
  - ***Model checking***
  - ***Simulation***
  - ***Testing***

Model Checking

- **Model checking is an automated technique that, given a finite-state model of a system and a formal property, systematically checks whether this property holds for (a given state in) that model.**
- **Model checking explores all possible system states in a brute-force manner.**
- ***it can be shown that a given system model truly satisfies a certain property.***

Model Checking

- **Qualitative Properties:**
  - **Is the generated result OK?**
  - **Can the system reach a deadlock situation?**
- ***Timing properties:***
  - ***Can a deadlock occur within 1 hour after a system reset?***
  - ***Is a response always received within 8 minutes?***

Model Checking

- **Model checking requires a precise and unambiguous statement of the properties to be examined.**
- **Making an accurate system model, this step often leads to the discovery of several ambiguities and inconsistencies in the informal documentation.**

Model Checking: Example

- **Concurrency and Atomicity**
- ***proc Inc = while true do if x < 200 then x := x + 1 fi od***
- ***proc Dec = while true do if x > 0 then x := x – 1 fi od***
- ***proc Reset = while true do if x = 200 then x := 0 fi od***

CS636 Formal Methods

Topics Covered

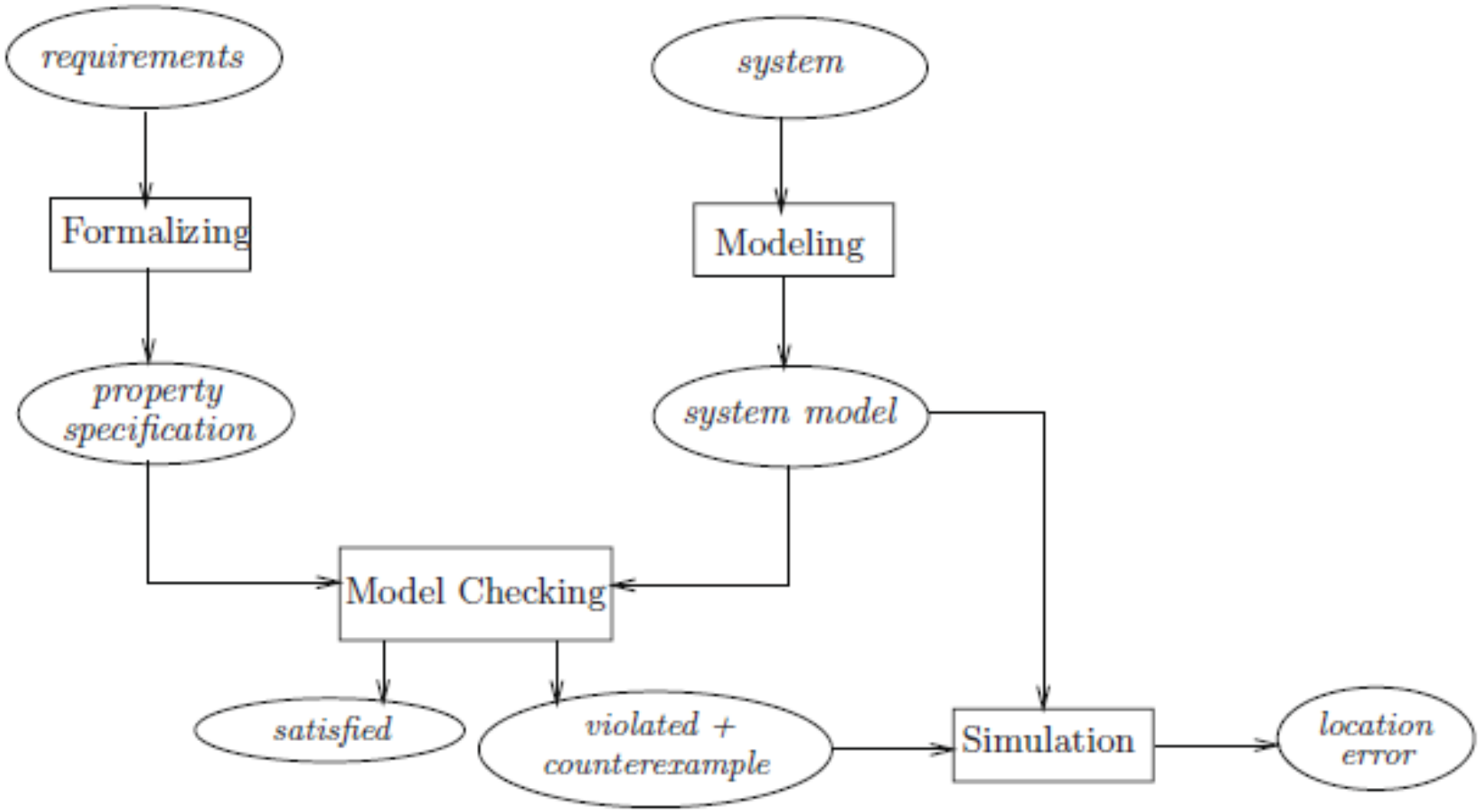
Formal Verification

References:

All the contents in these slides are taken from:

- Baier, Christel, and Joost-Pieter Katoen. *Principles of model checking*. MIT press, 2008.

The Model-Checking Process



The Model-Checking Process

- **Modeling phase:**
  - **– model the system under consideration using the model description language of the model checker at hand;**
  - **– as a first sanity check and quick assessment of the model perform some simulations;**
  - **– formalize the property to be checked using the property specification language.**
- **Running phase: run the model checker to check the validity of the property in the system model.**
- **Analysis phase:**
  - **– property satisfied? → check next property (if any);**
  - **– property violated? →**
    1. **analyze generated counterexample by simulation;**
    2. **refine the model, design, or property;**
    3. **repeat the entire procedure.**
- **– out of memory? → try to reduce the model and try again**

Strengths of Model checking

- **Wide range of applications**
- ***It supports partial verification***
- ***Its not vulnerable to likelihood that an error is exposed***
- ***It provides diagnostic information in case a property is invalidated***
- ***It is a potential "push-button" technology***
- ***It enjoys a rapidly increasing interest by industry;***

Strengths of Model checking

- **It can be easily integrated in existing development cycles;**
- ***It has a sound and mathematical underpinning;***

Weaknesses of Model checking

- **It is mainly appropriate to control-intensive applications and less suited for data intensive applications**
- ***Its applicability is subject to decidability issues;***
- ***It verifies a system model, and not the actual system (product or prototype) itself.***
- ***It checks only stated requirements***
- ***Its usage requires some expertise in finding appropriate abstractions to obtain smaller system models and to state properties in the logical formalism used***

Weaknesses of Model checking

- **It is not guaranteed to yield correct results: as with any tool, a model checker may contain software defects**
- ***It does not allow checking generalizations***

CS636 Formal Methods

Topics Covered

Transition Systems

References:

All the contents in these slides are taken from:

- Baier, Christel, and Joost-Pieter Katoen. *Principles of model checking*. MIT press, 2008.

Transition Systems

- **Transition systems are often used in computer science as models to describe the behavior of systems.**
- ***They are basically directed graphs where nodes represent states, and edges model transitions, i.e., state changes.***
- ***A state describes some information about a system at a certain moment of its behavior.***
- ***Transitions specify how the system can evolve from one state to another***

Transition Systems

- **A transition system TS is a tuple (S, Act,→,I, AP, L) where**
- ***S is a set of states,***
- ***Act is a set of actions,***
- ***→ ⊆ S × Act × S is a transition relation,***
- ***I ⊆ S is a set of initial states,***
- ***AP is a set of atomic propositions, and***
- ***L : S → 2^{AP} is a labeling function.*** ksi bi state ko ksi b propositionn k sth attach krta ha

Transition Systems

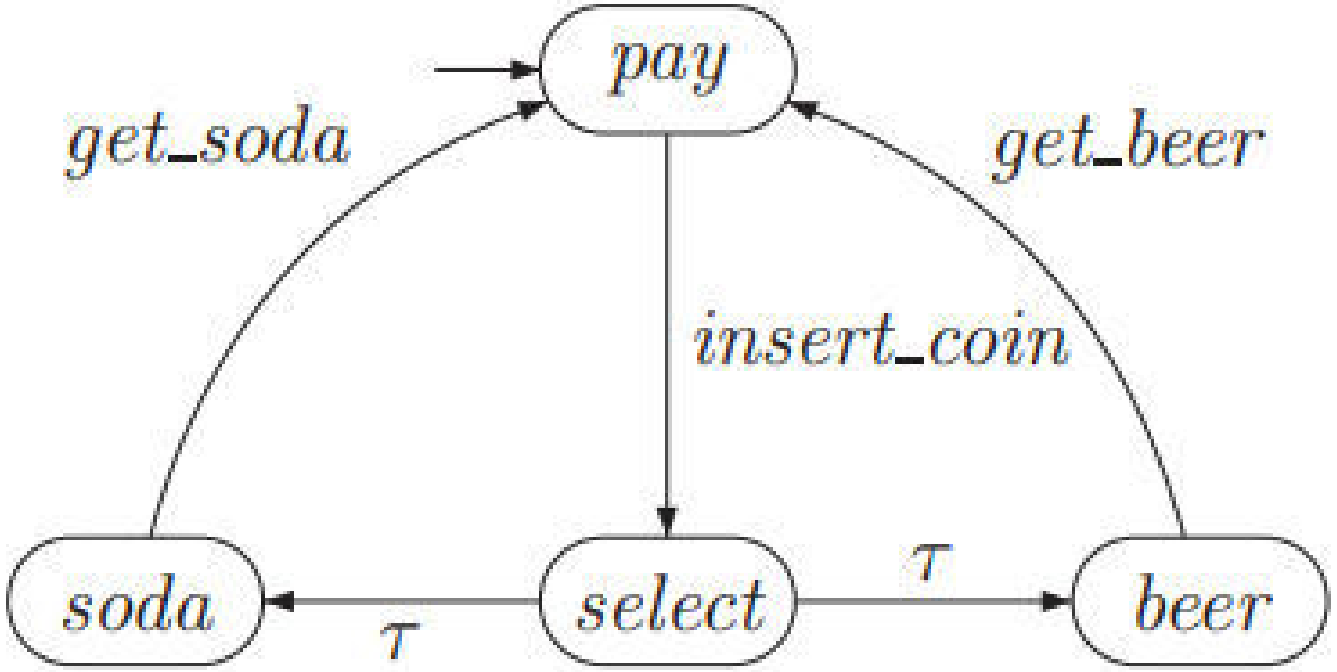
S = {pay,select,soda,beer}

Act = {insert_coin,get_soda,get_beer, τ}

I = {pay}

AP = S

L(s) = {s}



Transition Systems

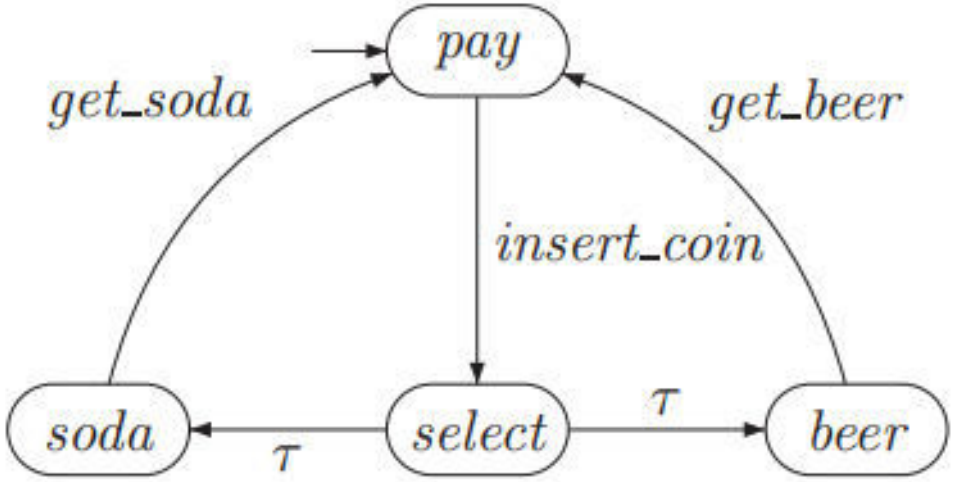
The vending machine only delivers a drink after providing a coin

AP = { paid , drink }

L(pay) = ∅, L(soda) = L(beer) = { paid , drink }, L(select) = { paid }.

Here, the proposition paid characterizes exactly those states in which the user has already paid but not yet obtained a beverage.

An atomic proposition is a statement or assertion that must be true or false.



Direct Predecessors and Successors

Let $TS = (S, Act, \rightarrow, I, AP, L)$ be a transition system. For $s \in S$ and $\alpha \in Act$, the set of direct α-*successors* of s is defined as:

$$Post(s, \alpha) = \left\{ s' \in S \mid s \xrightarrow{\alpha} s' \right\}, \quad Post(s) = \bigcup_{\alpha \in Act} Post(s, \alpha).$$

The set of α-*predecessors* of s is defined by:

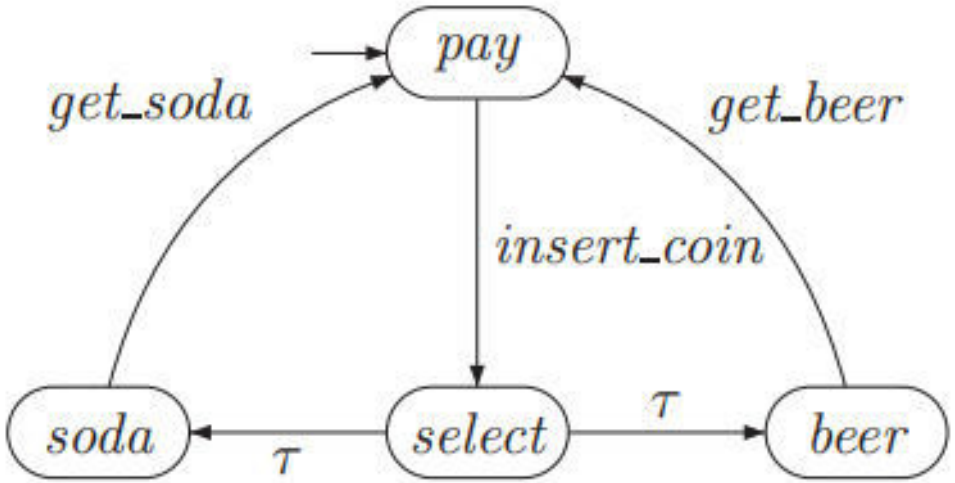
$$Pre(s, \alpha) = \left\{ s' \in S \mid s' \xrightarrow{\alpha} s \right\}, \quad Pre(s) = \bigcup_{\alpha \in Act} Pre(s, \alpha).$$

The Successor is defined as after numbers (that appear just after), and the Predecessor is known as before numbers (that occur just before).

Post(pay, insert_coin) = {select}

Pre(pay,get_soda) = {soda}

Pre(pay) = {soda;beer}



Terminal State

Question

State s in transition system TS is called terminal if and only if Post(s) = ∅.

Deterministic Transition System

Let TS = (S, Act, $\rightarrow$, I, AP, L) be a transition system.

TS is called action-deterministic if $|I| \leq 1$ and $|\text{Post}(s, \alpha)| \leq 1$ for all states s and actions α.

TS is called AP-deterministic if $|I| \leq 1$ and $|\text{Post}(s) \cap \{s' \in S | L(s') = A\}| \leq 1$ for all states s and actions $A \in 2^{AP}$.

Execution Fragment

Let TS = (S, Act,→, I,AP, L) be a transition system. A finite execution fragment ρ of TS is an alternating sequence of states and actions ending with a state

$\rho = s_0\alpha_1 s_1\alpha_2 \ldots \alpha_n s_n \; such\; that\; s_i \xrightarrow{\alpha_{i+1}} s_{i+1}$ for all $0 \leq i \leq n$

n= length of execution fragment

Finite and Infinite Execution Fragment

Finite execution fragment

$\rho = s_0\alpha_1 s_1\alpha_2 \ldots \alpha_n s_n \; such\ that\ s_i \xrightarrow{\alpha_{i+1}} s_{i+1}$ for all $0 \leq i \leq n$

Infinite Execution fragment

$\rho = s_0\alpha_1 s_1\alpha_2 \ldots \alpha_n s_n \; such\ that\ s_i \xrightarrow{\alpha_{i+1}} s_{i+1}$ for all $0 \leq i$

Maximal and Initial execution fragment

- A maximal execution fragment is either a finite execution fragment that ends in a terminal state, or an infinite execution fragment. An execution fragment is called initial if it starts in an initial state

$$\rho_1 = pay \xrightarrow{coin} select \xrightarrow{\tau} soda \xrightarrow{sget} pay \xrightarrow{coin} select \xrightarrow{\tau} soda \xrightarrow{sget} \ldots$$

$$\rho_2 = select \xrightarrow{\tau} soda \xrightarrow{sget} pay \xrightarrow{coin} select \xrightarrow{\tau} beer \xrightarrow{bget} \ldots$$

$$\varrho = pay \xrightarrow{coin} select \xrightarrow{\tau} soda \xrightarrow{sget} pay \xrightarrow{coin} select \xrightarrow{\tau} soda \ .$$

Reachable States

- Let TS = (S, Act,→, I,AP, L) be a transition system. A state s ∈ S is called reachable in TS if there exists an initial, finite execution fragment

$$s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} \ldots \xrightarrow{\alpha_n} s_n = s \ .$$

CS636 Formal Methods

Topics Covered

Concurrent Systems

References:

All the contents in these slides are taken from:

- Baier, Christel, and Joost-Pieter Katoen. *Principles of model checking*. MIT press, 2008.

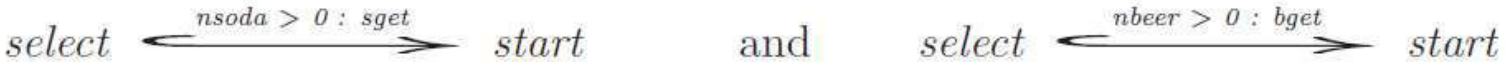
Data Dependent Systems

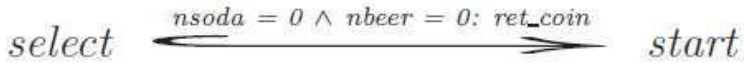
- **The executable actions of a data-dependent system typically result from conditional branching, as in**
  - **if x%2 = 1 then x := x + 1 else x := 2·x fi. In**



- Labels of conditional transitions are of the form $g : \alpha$ where g is a Boolean condition (called guard), and α is an action that is possible once g holds.

Data Dependent Systems





Data Dependent Systems

Action	Effect
refill	$nsoda := max;\ nbeer := max$
sget	$nsoda := nsoda - 1$
bget	$nbeer := nbeer - 1$

action previous new

- *Effect* : *Act* × *Eval*(*Var*) → *Eval*(*Var*)

- which indicates how the evaluation η of variables is changed by performing an action. If, e.g., α denotes the action $x := y+5$, where x and y are integer variables, and η is the evaluation with $\eta(x) = 17$ and $\eta(y) = -2$, then
- $Effect(\alpha, \eta)(x) = \eta(y) + 5 = -2 + 5 = 3$, and $Effect(\alpha, \eta)(y) = \eta(y) = -2$.

Program Graph

A *program graph* PG over set Var of typed variables is a tuple $(Loc, Act, Effect, \hookrightarrow, Loc_0, g_0)$ where

- Loc is a set of locations and Act is a set of actions,
- $Effect : Act \times Eval(Var) \to Eval(Var)$ is the effect function,
- $\hookrightarrow \subseteq Loc \times Cond(Var) \times Act \times Loc$ is the conditional transition relation,
- $Loc_0 \subseteq Loc$ is a set of initial locations,
- $g_0 \in Cond(Var)$ is the initial condition.

Program Graph Example

- *Var = { nsoda, nbeer }*
- *Act = { bget , sget, coin, ret coin, refill } .*
- *Effect*(*coin, η*) = *η*
- *Effect*(*ret_coin, η*) = *η*
- *Effect*(*sget, η*) = *η*[*nsoda* := *nsoda*−1]
- *Effect*(*bget, η*) = *η*[*nbeer* := *nbeer*−1]
- *Effect*(*refill, η*) = [*nsoda* := *max, nbeer* := *max*]

Program Graph

The transition system $TS(PG)$ of program graph

$$PG = (Loc, Act, Effect, \hookrightarrow, Loc_0, g_0)$$

over set Var of variables is the tuple $(S, Act, \longrightarrow, I, AP, L)$ where

- $S = Loc \times Eval(Var)$
- $\longrightarrow \subseteq S \times Act \times S$ is defined by the following rule (see remark below):

$$\frac{\ell \overset{g:\alpha}{\hookrightarrow} \ell' \quad \wedge \quad \eta \models g}{\langle \ell, \eta \rangle \xrightarrow{\alpha} \langle \ell', Effect(\alpha, \eta) \rangle}$$

Premise

conclusion

- $I = \{\langle \ell, \eta \rangle \mid \ell \in Loc_0, \eta \models g_0\}$
- $AP = Loc \cup Cond(Var)$
- $L(\langle \ell, \eta \rangle) = \{\ell\} \cup \{g \in Cond(Var) \mid \eta \models g\}$.

CS636 Formal Methods

Topics Covered

Modeling Concurrent Systems

References:

All the contents in these slides are taken from:

- Baier, Christel, and Joost-Pieter Katoen. *Principles of model checking*. MIT press, 2008.

Inter-Diagram: Spans multiple diagrams, ensuring consistency across them.

Parallelism and Communication

- **Most hard- and software systems are not sequential but parallel in nature.**
- **Let us assume that the operational (stepwise) behavior of the processes that run in parallel are given by transition systems $TS_1, \ldots, TS_n$.**
- **The goal is to define an operator ||, such that:**
  - **$TS = TS_1 \,||\, TS_2 \,||\, \ldots \,||\, TS_n$**

Parallelism and Communication

- **The nature of the operator || will, of course, depend on the kind of communication that is supported.**
- **Note that the above scheme may be repeated for TSi, i.e., TS_i may again be a transition system that is composed of several transition systems:**
  - $TS_i = TS_{i,1} \,||\, TS_{i,2} \,||\, \ldots \,||\, TS_{i,ni}$.

acting together

to put layers or flat pieces of something between layers or flat pieces of something else

Concurrency and Interleaving

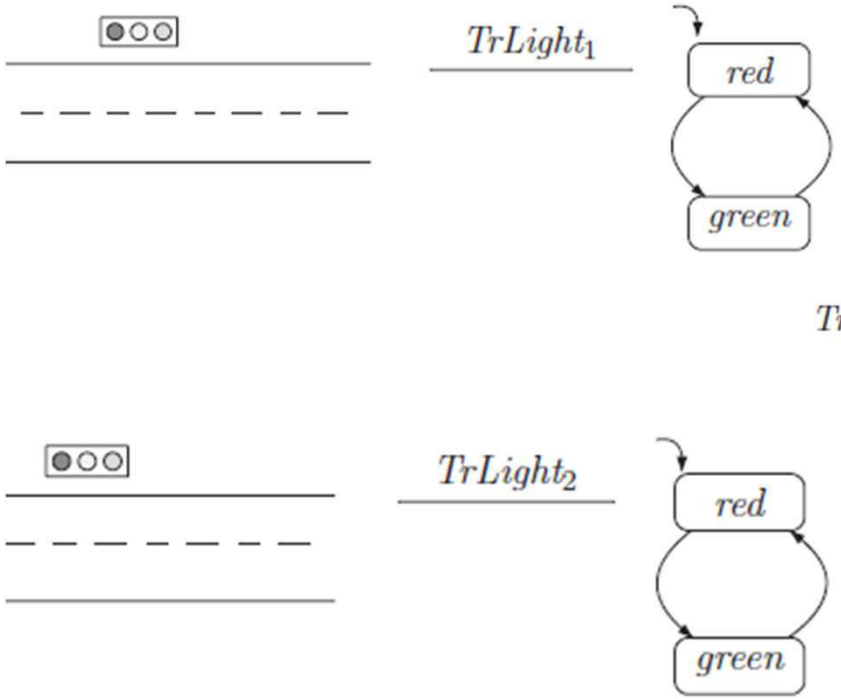
- **Widely adopted paradigm for parallel systems is that of interleaving.**
- **The global system state – composed of the current individual states of the components – plays a key role in interleaving.**
- **Actions of an independent component are merged (also called weaved), or "interleaved", with actions from other components.**

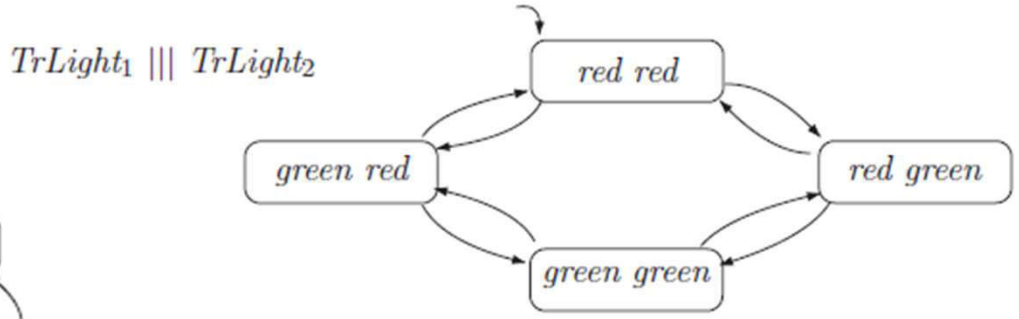
Concurrency and Interleaving

$$
\begin{array}{ccccccccccc}
P & Q & P & Q & P & Q & Q & Q & P & \ldots \\
P & P & Q & P & P & Q & P & P & Q & \ldots \\
P & Q & P & P & Q & P & P & P & Q & \ldots
\end{array}
$$

- An important justification for interleaving is the fact that the effect of concurrently executed, independent actions α and β, say, is identical to the effect when α and β are successively executed in arbitrary order. This can symbolically be stated as:
  - Effect(α ||| β, η) = Effect((α ; β) + (β ; α), η)

Concurrency and Interleaving

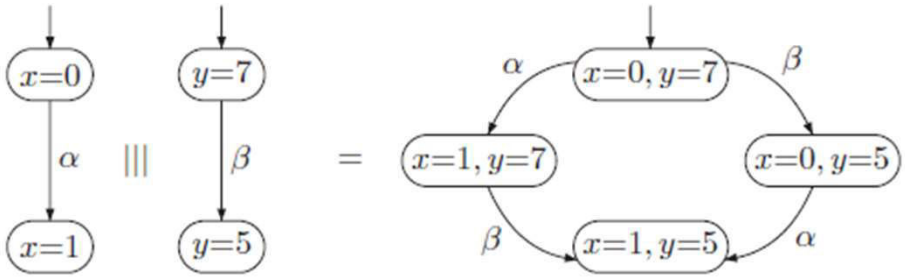




Concurrency and Interleaving

- Effect(α ||| β, η) = Effect((α ; β) + (β ; α), η)

$$\underbrace{x := x + 1}_{=\alpha} \;|||\; \underbrace{y := y - 2}_{=\beta}.$$



Interleaving of Transition Systems

- The transition system $TS_1 \,|||\, TS_2$ represents a parallel system resulting from the weaving (or merging) of the actions of the components as described by TS_1 and TS_2.
- It is assumed that no communication and no contentions (on shared variables) occur at all.
- The ("global") states of $TS_1 \,|||\, TS_2$ are pairs $\langle s_1, s_2 \rangle$ consisting of "local" states s_i of the components TS_i.
- The outgoing transitions of the global state $\langle s_1, s_2 \rangle$ consist of the outgoing transitions of s_1 together with those of s_2.

Interleaving of Transition Systems

Let $TS_i = (S_i, Act_i, \rightarrow_i, I_i, AP_i, L_i)$ i=1, 2, be two transition systems. The transition system $TS_1 \,|||\, TS_2$ is defined by:

$$TS_1 \,|||\, TS_2 = (S_1 \times S_2, Act_1 \cup Act_2, \rightarrow, I_1 \times I_2, AP_1 \cup AP_2, L)$$

where the transition relation $\rightarrow$ is defined by the following rules:

$$\frac{s_1 \xrightarrow{\alpha}_1 s_1'}{\langle s_1, s_2\rangle \xrightarrow{\alpha} \langle s_1', s_2\rangle} \quad \text{and} \quad \frac{s_2 \xrightarrow{\alpha}_2 s_2'}{\langle s_1, s_2\rangle \xrightarrow{\alpha} \langle s_1, s_2'\rangle}$$

and the labeling function is defined by $L(\langle s_1, s_2\rangle) = L(s_1) \cup L(s_2)$.

Interleaving of Transition Systems

Let $PG_i = (Loc_i, Act_i, Effect_i, \hookrightarrow_i, Loc_{0,i}, g_{0,i})$, for $i{=}1,2$ be two program graphs over the variables Var_i. Program graph $PG_1 \,|||\, PG_2$ over $Var_1 \cup Var_2$ is defined by

$$PG_1 \,|||\, PG_2 \;=\; (Loc_1 \times Loc_2, Act_1 \uplus Act_2, Effect, \hookrightarrow, Loc_{0,1} \times Loc_{0,2}, g_{0,1} \wedge g_{0,2})$$

where $\hookrightarrow$ is defined by the rules:

$$\frac{\ell_1 \overset{g:\alpha}{\hookrightarrow_1} \ell_1'}{\langle \ell_1, \ell_2 \rangle \overset{g:\alpha}{\hookrightarrow} \langle \ell_1', \ell_2 \rangle} \quad \text{and} \quad \frac{\ell_2 \overset{g:\alpha}{\hookrightarrow_2} \ell_2'}{\langle \ell_1, \ell_2 \rangle \overset{g:\alpha}{\hookrightarrow} \langle \ell_1, \ell_2' \rangle}$$

and $Effect(\alpha, \eta) \;=\; Effect_i(\alpha, \eta)$ if $\alpha \in Act_i$.

CS636 Formal Methods

Topics Covered

Analyzing Well formedness

References:

All the contents in these slides are taken from:

- Zhu, H., & Shan, L. (2006). Well-formedness, consistency and completeness of graphic models. Proc. of UKSIM, 6, 47-53.

Well formedness

- **A model or a diagram is well-formed if its structure satisfies the syntactic rules, especially its elements are type compatible if a type system is defined.** A type system is a set of rules that define and enforce the structure and organization of data, ensuring consistency and accuracy.
- **A Modeling Language: ML**
- **Model: M**
- **Model M has Diagrams $D \neq \phi$**
- **Type of Diagram: $T \neq \phi$**
- **T-Submodel= The subset of diagrams of type T in a model M = T.M**

Condition

Consistency Constraints

- **Generally speaking, a consistency constraint C is a predicate defined on models such that C(M) = true means that the model is consistent with respect to the constraint;**

Consistency Constraints: Intra diagram

- **A consistency constraint C is called intra-diagram, if it is defined on a specific type T of diagram of the model in the form of: C(M) ⇔ ∀D∈M.T. C'(D)**

Intra-Diagram: Operates within a single diagram (e.g., ensuring unique class names in a single UML class diagram).

Type text here

Consistency Constraints: Inter diagrams

- **A consistency constraint C is inter-diagram, if it is defined on two or more diagrams.**
- **For example, a consistency constraint C that is defined on two diagrams of type T is an inter-diagram consistency constraint, where**

$$C(M) \Leftrightarrow \forall D,D' \in M.T.C'(D, D'),$$

Inter-Diagram: Spans multiple diagrams, ensuring consistency across them.

Consistency Constraints: Inter Model

- **A consistency constraint C is called inter-model, if it is defined on diagrams of more than one type, say between diagrams of types T1 and T2, so that:**

$$C(M) \Leftrightarrow \forall D \in M.T_1,\ D' \in M.T_2.C'(D, D')$$

Comparison of Constraint Types

Constraint Type	Scope	Example
Intra-Diagram	Within a single diagram.	Unique names in a UML Class Diagram.
Inter-Diagram	Between diagrams of the same type.	Consistency between two Class Diagrams.
Inter-Model	Between diagrams of different types.	Alignment between a Class Diagram and a Sequence Diagram.

Completeness Constraints

- **A completeness constraint restricts the construction of the models so that certain types of errors due to the lack of information can be prevented and detected.**

$$\forall \boldsymbol{D} \in \boldsymbol{M_{uc}}, \forall \boldsymbol{X} \in \boldsymbol{D}, \exists \boldsymbol{AD} \in \boldsymbol{M_{AD}}, \boldsymbol{title}(\boldsymbol{X}) = \boldsymbol{title}(\boldsymbol{AD})$$

Key Differences:

Aspect	Use Case Diagram	Activity Diagram
Purpose	Represents what the system does (high-level functions).	Describes how a specific process or task is performed (detailed).
Focus	Interaction between users and the system (external view).	Internal flow of activities in a process (dynamic behavior).
Elements	Actors, use cases, relationships.	Activities, transitions, decisions, start/end points.
Level of Detail	High-level, abstract.	Detailed, showing specific steps and decision points.
Example	User interacting with the ATM to withdraw cash.	Steps involved in withdrawing cash: entering the PIN, selecting the amount, etc.

CS636 Formal Methods

Topics Covered

Formal Reasoning

Formal Reasoning

- It is a process of thinking, logically arguing and drawing conclusions
- It involves the application of well-defined rules, principles, and procedures to analyze and draw conclusions about a given problem or situation.
- A systematic approach to logical thinking and problem-solving using formal systems, such as mathematical or logical frameworks.
- Logic is a language of reasoning.

Logic

- Logic refers to a formal system of principles and rules that govern correct or valid reasoning.
- Logic uses symbols, notation, and formal languages to represent statements and employs logical operators and rules of inference to derive conclusions.
- It provides a framework for evaluating the logical coherence and validity of arguments.
- Application in computer science – design computer circuits, developments of computer programs, verification of the correctness of programs.
- Providing such a rigorous framework makes it possible to reason precisely about statements made in the natural language

Propositions

- Propositions are statements that are either TRUE or FALSE.
- The following are examples of propositions that evaluate to TRUE:
  - There are seven days in a week
  - There are 12 months in a year
  - 2 + 4 = 6
- The following propositions evaluate to FALSE:
  - The angles of a triangle add up to 360 degree
  - London is the capital of France
  - 2 - 4 = 7

Reference: Formal Software Development From VDM to Java, by Quentin Charatan and Aaron Kans

Propositions

- In mathematics we often represent a proposition symbolically by a variable name
- such as P or Q
- For example:
  - P: I go shopping on Wednesdays
  - Q: 102.001 > 101.31

Reference: Formal Software Development From VDM to Java, by Quentin Charatan and Aaron Kans

Propositions

- In the worlds of computing and mathematics, occasions arise when it is not possible to evaluate expressions precisely – maybe a program terminated incorrectly, or perhaps somebody tried to evaluate the square root of a negative integer.
- it is possible to account for such situations by defining a three-valued logic, which allows a proposition to take the value UNDEFINED as well as TRUE or FALSE.

Reference: Formal Software Development From VDM to Java, by Quentin Charatan and Aaron Kans

CS636 Formal Methods

Topics Covered

Propositional Logic

References

- All the contents of this video lecture and PPT slides are taken from:

Theory and Problems of DISCRETE MATHEMATICS, Third Edition, by SEYMOUR LIPSCHUTZ and MARC LARS LIPSON

Propositions

- Propositions are statements that are either TRUE or FALSE.
- The following are examples of propositions that evaluate to TRUE:
  - There are 12 months in a year
  - 3 + 6 = 9
- The following propositions evaluate to FALSE:
  - Lahore is Capital of Pakistan
  - 4-7=36
- In mathematics we often represent a proposition symbolically by a variable name
- such as P or Q
- For example:
  - P: 3+6=9
  - Q: I am a citizen of Pakistan

Compound Propositions

- Many propositions are composite, that is, composed of sub-propositions and various connectives. Such composite propositions are called compound propositions.
- A proposition is said to be primitive if it cannot be broken down into simpler propositions, that is, if it is not composite.
- Primitive: 2+2=4
- Primitive: Ice floats in water
- Compound: Roses are red and violets are blue

Compound Propositions

- The fundamental property of a compound proposition is that its truth value is completely determined by the truth values of its subpropositions together with the way in which they are connected to form the compound propositions.

Basic Logical Operations

- **Conjunction:** Corresponds to English word “and”
- Any two propositions can be combined by the word “and” to form a compound proposition called the conjunction of the original propositions. Symbolically:
- $p \wedge q$
- Definition: If p and q are true, then $p \wedge q$ is true; otherwise $p \wedge q$ is false.

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

Basic Logical Operators (Conjunction)

P ∧ Q

1. Ice floats in water and 2 + 2 = 4.
2. Ice floats in water and 2 + 2 = 5.
3. China is in Europe and 2 + 2 = 4.
4. China is in Europe and 2 + 2 = 5.

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

- Only the first statement is true. Each of the others is false since at least one of its sub statements is false.

Basic Logical Operators

- **Disjunction:** Corresponds to English word "or"
- Any two propositions can be combined by the word "or" to form a compound proposition called the disjunction of the original propositions. Symbolically:
- $p \vee q$
- Definition: If p and q are false, then $p \vee q$ is false; otherwise $p \vee q$ is true.

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Basic Logical Operators (Disjunction)

1. Ice floats in water or 2 + 2 = 4.
2. Ice floats in water or 2 + 2 = 5.
3. China is in Europe or 2 + 2 = 4.
4. China is in Europe or 2 + 2 = 5.

- Only the last statement (iv) is false. Each of the others is true since at least one of its sub-statements is true.

Basic Logical Operators

- **Negation:** Corresponds to English word "not"
- Given any proposition p, another proposition, called the negation of p, can be formed by writing "It is not true that . . ." or "It is false that. . ." before p or, if possible, by inserting in p the word "not." Symbolically: $\neg p$
- Definition: If p is true, then ¬p is false; and if p is false, then ¬p is true.

p	$\neg p$
T	F
F	T

Basic Logical Operators (Negation)

(*a*1) Ice floats in water.

(*a*2) It is false that ice floats in water.

(*a*3) Ice does not float in water.

(*b*1) 2 + 2 =5

(*b*2) It is false that 2 + 2 = 5.

(*b*3) 2 + 2 ≠ 5

(a2) and (a3) are each the negation of (a1); and (b2) and (b3) are each the negation of (b1). Since (a1) is true, (a2) and (a3) are false; and since (b1) is false, (b2) and (b3) are true.

CS636 Formal Methods

Topics Covered

Propositional Logic

References

- All the contents of this video lecture and PPT slides are taken from:

Theory and Problems of DISCRETE MATHEMATICS, Third Edition, by SEYMOUR LIPSCHUTZ and MARC LARS LIPSON

PROPOSITIONS AND TRUTH TABLES

- Let P(p, q, . . .) denote an expression constructed from logical variables p, q, . . ., and the logical connectives ∧, ∨, and ¬, Such an expression P(p, q, . . .) will be called a proposition.
- The truth value of a proposition is known once the truth value of each of its variables is known.

How to construct a Truth Table

- Consider, for example, the proposition ¬(p∧¬q). Following is the truth table of ¬(p∧¬q)

p	q	$\neg q$	$p \wedge \neg q$	$\neg(p \wedge \neg q)$
T	T	F	F	T
T	F	T	T	F
F	T	F	F	T
F	F	T	F	T

- First columns of the table are for the variables p, q, . . . and that there are enough rows in the table, to allow for all possible combinations of T and F for these variables. (For 2 variables, as above, 4 rows are necessary; for 3 variables, 8 rows are necessary; and, in general, for n variables, 2^n rows are required.)

How to construct a Truth Table

p	q	$\neg q$	$p \wedge \neg q$	$\neg(p \wedge \neg q)$
T	T	F	F	T
T	F	T	T	F
F	T	F	F	T
F	F	T	F	T

- There is then a column for each “elementary” stage of the construction of the proposition, the truth value at each step being determined from the previous stages by the definitions of the connectives ∧, ∨, ¬.
- Finally we obtain the truth value of the proposition, which appears in the last column.

How to construct a Truth Table (alternate method)

p	q	$\neg$	$(p$	$\wedge$	$\neg$	$q)$
T	T					
T	F					
F	T					
F	F					.
Step						

p	q	$\neg$	$(p$	$\wedge$	$\neg$	$q)$
T	T		T		F	T
T	F		T		T	F
F	T		F		F	T
F	F		F		T	F
Step			1		2	1

(*a*)

p	q	$\neg$	$(p$	$\wedge$	$\neg$	$q)$
T	T		T	F	F	T
T	F		T	T	T	F
F	T		F	F	F	T
F	F		F	F	T	F
Step			1	3	2	1

(*b*)

p	q	$\neg$	$(p$	$\wedge$	$\neg$	$q)$
T	T	T	T	F	F	T
F	T	F	T	T	T	F
F	F	T	F	F	F	T
F	F	T	F	F	T	F
Step		4	1	3	2	1

(*c*)

p	q	$\neg(p \wedge \neg q)$
T	T	T
T	F	F
F	T	T
F	F	T

LOGICAL EQUIVALENCE

- Two propositions P(p, q, . . .) and Q(p, q, . . .) are said to be logically equivalent, or simply equivalent or equal, if they have identical truth tables.
- Denoted by: P(p, q, . . .) ≡ Q(p, q, . . .)
- ¬(p ∧ q) ≡ ¬p ∨¬q

p	q	$p \wedge q$	$\neg(p \wedge q)$
T	T	T	F
T	F	F	T
F	T	F	T
F	F	F	T

(a) $\neg(p \wedge q)$

p	q	$\neg p$	$\neg q$	$\neg p \vee \neg q$
T	T	F	F	F
T	F	F	T	T
F	T	T	F	T
F	F	T	T	T

(b) $\neg p \vee \neg q$

LOGICAL EQUIVALENCE

- $\neg(p \wedge q) \equiv \neg p \vee \neg q$
- Let p be “Roses are red” and q be “Violets are blue.”
- Let S be the statement: “It is not true that roses are red and violets are blue.”
- Then S = $\neg(p \wedge q)$.
- However, as noted above, $\neg(p \wedge q) \equiv \neg p \vee \neg q$. Accordingly, S has the same meaning as the statement:
- “Roses are not red, or violets are not blue.”

CS636 Formal Methods

Topics Covered

Propositional Logic

References

- All the contents of this video lecture and PPT slides are taken from:

Theory and Problems of DISCRETE MATHEMATICS, Third Edition, by SEYMOUR LIPSCHUTZ and MARC LARS LIPSON

Laws of the algebra of propositions

Idempotent laws:	(1a) $p \vee p \equiv p$	(1b) $p \wedge p \equiv p$
Associative laws:	(2a) $(p \vee q) \vee r \equiv p \vee (q \vee r)$	(2b) $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
Commutative laws:	(3a) $p \vee q \equiv q \vee p$	(3b) $p \wedge q \equiv q \wedge p$
Distributive laws:	(4a) $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$	(4b) $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
Identity laws:	(5a) $p \vee F \equiv p$ (6a) $p \vee T \equiv T$	(5b) $p \wedge T \equiv p$ (6b) $p \wedge F \equiv F$
Involution law:	(7) $\neg\neg p \equiv p$	
Complement laws:	(8a) $p \vee \neg p \equiv T$ (9a) $\neg T \equiv F$	(8b) $p \wedge \neg p \equiv F$ (9b) $\neg F \equiv T$
DeMorgan's laws:	(10a) $\neg(p \vee q) \equiv \neg p \wedge \neg q$	(10b) $\neg(p \wedge q) \equiv \neg p \vee \neg q$

CS636 Formal Methods

Topics Covered

Propositional Logic

References

- All the contents of this video lecture and PPT slides are taken from:

- Formal Software Development From VDM to Java, by Quentin Charatan and Aaron Kans

The implication operator

- In defining an implication operator we attempt to give meaning to the expression P implies Q.
- The implication operator is represented by the symbol ⇒.
- The statement P implies Q is therefore represented by: P⇒Q
- An alternative way of expressing implication is *if P then Q.*
- Thus if P represented the statement *It is rainy day* and Q represented the statement *I go the outdoors* then P⇒Q would represent the statement *If it is rainy day I go the outdoors*.

The implication operator (Truth table)

P	Q	$P \Rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

The equivalence operator

- It is represented by the symbol ⇔.
- It states: if P is TRUE then Q is TRUE, otherwise Q is FALSE; in other words, P is equivalent to Q, which is represented by: P⇔Q

p if and only if q

Equivalence operator true tb hota ha jb dono P aur Q apas mn equivalent ho

P	Q	P ⇔ Q
T	T	T
T	F	F
F	T	F
F	F	T

The exclusive or operator

- The ‘or’ operator evaluates to TRUE if either of p or q or both are true.
- The natural language ‘or’ usually implies that only one or other of the statements is TRUE but not both.
- This corresponds to the logical operator known as exclusive or (sometimes referred to as xor), which is represented by the symbol: $\oplus$.

Is mein ak time per ak e condition true hoskti ha

P	Q	$P \oplus Q$
T	T	F
T	F	T
F	T	T
F	F	F

CS636 Formal Methods

Topics Covered

Propositional Logic

References

- All the contents of this video lecture and PPT slides are taken from:

Theory and Problems of DISCRETE MATHEMATICS, Third Edition, by SEYMOUR LIPSCHUTZ and MARC LARS LIPSON

TAUTOLOGIES AND CONTRADICTIONS

- Some propositions P(p, q, . . .) contain only T in their evaluation or, in other words, they are true for any truth values of their variables. Such propositions are called tautologies.
- Analogously, a proposition P(p, q, . . .) is called a contradiction if it contains only F in its evaluation or, in other words, if it is false for any truth values of its variables.

p	$\neg p$	$p \vee \neg p$
T	F	T
F	T	T

p	$\neg p$	$p \wedge \neg p$
T	F	F
F	T	F

ARGUMENTS

- An argument is an assertion that a given set of propositions P_1, P_2,...,P_n, yields another proposition Q.
- An argument provides support or evidence in favor of one of the others.
- An argument P_1, P_2, ..., $P_n \vdash Q$ is said to be valid if Q is TRUE whenever all the premises P_1, P_2,...,P_n are true.
- Eg: valid argument

$$p, p \rightarrow q \vdash q$$

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

ARGUMENTS

- Now the propositions P1, P2, . . . , Pn are true simultaneously if and only if the proposition P1 ∧ P2 ∧ . . . Pn is true.
- Thus the argument P1, P2, . . . , Pn ⊢ Q is valid if and only if Q is true whenever P1 ∧ P2 ∧ . . . ∧ Pn is true, or, equivalently, if the proposition (P1 ∧ P2 ∧ . . . ∧ Pn) → Q is a tautology.

ARGUMENTS

Lets determine the validity of the following argument:

p → q, ¬ q ⊢ ¬p.

p	q	$[(p$	$\rightarrow$	$q)$	$\wedge$	$\neg$	$q]$	$\rightarrow$	$\neg$	p
T	T	T	T	T	F	F	T	T	F	T
T	F	T	F	F	F	T	F	T	F	T
F	T	F	T	T	F	F	T	T	T	F
F	F	F	T	F	T	T	F	T	T	F
Step		1	2	1	3	2	1	4	2	1

ARGUMENTS

- An argument which is not valid is called fallacy.
- Eg: fallacy argument
- $p \rightarrow q, q \vdash p$

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

ARGUMENTS

Show that the following argument is a fallacy: p → q, ¬p ⊢ ¬q.

p	q	$p \rightarrow q$	$\neg p$	$(p \rightarrow q) \wedge \neg p$	$\neg q$	$[(p \rightarrow q) \wedge \neg p] \rightarrow \neg q$
T	T	T	F	F	F	T
T	F	F	F	F	T	T
F	T	T	T	T	F	F
F	F	T	T	T	T	T

CS636 Formal Methods

Topics Covered

Propositional Logic

References

- All the contents of this video lecture and PPT slides are taken from:

- Formal Software Development From VDM to Java, by Quentin Charatan and Aaron Kans

THREE-VALUED LOGIC

- Classical logic assumes that all expressions evaluate to TRUE or FALSE.
- In reality, this is not always the case when evaluating an expression, because sometimes an expression can be undefined
  - For example, the expression 0/0.
- Undefined terms are very common in programming situations – for example, when a variable is first declared and has not yet been assigned a value.
- In this system of logic a proposition could have the value TRUE, FALSE or UNDEFINED.

AND Operator

P	Q	$P \wedge Q$
T	T	**T**
T	F	**F**
T	UNDEFINED	**UNDEFINED**
F	T	**F**
F	F	**F**
F	UNDEFINED	**F**
UNDEFINED	T	**UNDEFINED**
UNDEFINED	F	**F**
UNDEFINED	UNDEFINED	**UNDEFINED**

OR Operator

P	*Q*	*P* ∨ *Q*
T	T	**T**
T	F	**T**
T	UNDEFINED	**T**
F	T	**T**
F	F	**F**
F	UNDEFINED	**UNDEFINED**
UNDEFINED	T	**T**
UNDEFINED	F	**UNDEFINED**
UNDEFINED	UNDEFINED	**UNDEFINED**

IMPLICATION Operator

P	Q	$P \Rightarrow Q$
T	T	**T**
T	F	**F**
T	UNDEFINED	**UNDEFINED**
F	T	**T**
F	F	**T**
F	UNDEFINED	**T**
UNDEFINED	T	**T**
UNDEFINED	F	**UNDEFINED**
UNDEFINED	UNDEFINED	**UNDEFINED**

Equivalence Operator

P	Q	$P \Leftrightarrow Q$
T	T	**T**
T	F	**F**
T	UNDEFINED	**UNDEFINED**
F	T	**F**
F	F	**T**
F	UNDEFINED	**UNDEFINED**
UNDEFINED	T	**UNDEFINED**
UNDEFINED	F	**UNDEFINED**
UNDEFINED	UNDEFINED	**UNDEFINED**

Exclusive OR Operator

P	Q	$P \oplus Q$
T	T	**F**
T	F	**T**
T	UNDEFINED	**UNDEFINED**
F	T	**T**
F	F	**F**
F	UNDEFINED	**UNDEFINED**
UNDEFINED	T	**UNDEFINED**
UNDEFINED	F	**UNDEFINED**
UNDEFINED	UNDEFINED	**UNDEFINED**

Negation Operator

P	$\neg P$
T	F
F	T
UNDEFINED	UNDEFINED

CS636 Formal Methods

Topics Covered

Predicate Logic

References

- All the contents of this video lecture and PPT slides are taken from:

- Formal Software Development From VDM to Java, by Quentin Charatan and Aaron Kans

Predicate Logic

- One of the limitations with the propositional logic is that, while it allows us to argue about individual values, it does not give us the ability to argue about sets of values.
- A set is any well-defined, unordered, collection of objects.
  - The set of whole numbers from 1 to 10
  - The set of the days of the week
- In mathematics, we often represent a set elements by lower-case letters. For example:
- A = {s, d, f, h, k}
- B = {a, b, c, d, e, f}
- The symbol ∈ means ‘is an element of’. Therefore the statement ‘d is an element of A’ is written: d ∈ A

Predicate Logic

- For the purpose of reasoning about sets of values, a more powerful tool than the propositional logic has been devised, namely the predicate logic.
- A predicate is a truth-valued expression containing free variables. These allow the expression to be evaluated by giving different values to the variables.
- Once the variables are evaluated they are said to be bound.

Example of predicates

- Predicates can be named with either a single letter, or with a word that expresses the meaning of the predicate; the variables are placed in brackets after the name.
- This is made clear in the following examples:
  - C(x): x is a cat
  - Studies(x,y): x studies y
  - Prime(n): n is a prime number
- A statement such as C(x) can be read C of x.
- Predicates such as those above do not yet have a value – they only have a value when the variables themselves are given a value.

Binding variables

- Predicates do not yet have a value – they only have a value when the variables themselves are given a value.
- Two ways:
  - Substitution
  - Quantification

Binding (Substitution)

- For example, using the previous three predicates:

- C(Simba): Simba is a cat
- Studies(Ali, physics): Ali studies physics
- Prime(3): 3 is a prime number

The above expressions now have a value of TRUE or FALSE.

Binding (Quantification)

- A quantifier is a mechanism for specifying an expression about a set of values. There are three quantifiers that we can use, each with its own symbol:
- Universal Quantifier
- Existential Quantifier
- Unique Existential Quantifier

Quantification

- The universal quantifier ∀:
- This quantifier enables a predicate to make a statement about all the elements in a particular set. For example, if M(x) is the predicate x chases mice, we could write:
- ∀x ∈ Cats ● M(x)
- This reads *For all the x which are members of the set Cats, x chases mice*, or, more simply, *All cats chase mice*.

Quantification

- – The existential quantifier ∃
- In this case, a statement is made about whether or not at least one element of a set meets a particular criterion.
- For example, if, as above, P(n) is the predicate *n is a prime number*, then we could write:
- $\exists n \in \mathbb{N} \bullet P(n)$
- This reads *There exists an n in the set of natural numbers such that n is a prime number*, or, put another way, *There exists at least one prime number in the set of natural numbers*.

Quantification

- –The unique existential quantifier ∃!
- This quantifier modifies a predicate to make a statement about whether or not precisely one element of a set meets a particular criterion.
- For example, if G(x) is the predicate *x is green*, we could write
- ∃!x ∈ Cats ● G(x)
- This would mean *There is one and only one cat that is green*.